



Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavre

A Project Report
on
“Self-Healing Firmware Architecture for Fault-Tolerant
Embedded Systems using In-Application Programming”

[Code No.: COMP 313]
(For partial fulfillment of Year III / Semester II in Computer Science)

Submitted by
Rajat Dahal (034816-23)
Suvesh Gurung (034824-23)
Pramisha Sapkota (034862-23)
Nischal Subedi (034874-23)

Submitted to
Suman Shrestha
Department of Computer Science and Engineering

August 4, 2026

Bona fide Certificate

**This project work on
“Self-Healing Firmware Architecture for Fault-Tolerant Embedded
Systems using In-Application Programming”**

is the bona fide work of

“

**Rajat Dahal (034816-23),
Suvesh Gurung (034824-23),
Pramisha Sapkota (034862-23),
Nischal Subedi (034874-23)**

”

who carried out the project work under my supervision.

Project Supervisor

Sushil Nepal

Assistant Professor

Department of Computer Science and Engineering

Acknowledgements

This project would not have been possible without the invaluable support and constant guidance of our supervisor, **Mr. Sushil Nepal**. We are deeply grateful for his technical insights, patient mentorship, and the encouragement he provided at every stage of the development of **”Self-Healing Firmware Architecture for Fault-Tolerant Embedded Systems using In-Application Programming”**.

We would also like to express our sincere gratitude to the **Department of Computer Science and Engineering** at Kathmandu University (KU) for providing the necessary resources and a conducive environment for this research.

We also extend our thanks to the everyone who had worked on this domain, hence enabling us to have a head start for this project.

Abstract

Embedded systems deployed in the field are often susceptible to firmware corruption caused by environmental factors, electromagnetic interference, or "bit flips" in flash memory. This project proposes a self-healing mechanism for the STM32F103-C8T6 (Blue Pill) module to ensure high reliability. The purpose is to develop a system that can autonomously detect and repair corrupted application code without external intervention. This will be implemented by partitioning the internal flash memory to store a backup copy of the original firmware. The procedure involves writing a custom bootloader that utilizes In-Application Programming (IAP) and Cyclic Redundancy Check (CRC) to verify firmware integrity during every boot cycle. If corruption is detected, the bootloader will overwrite the active application partition with the backup copy. The expected outcome is a resilient microcontroller system that remains operable even after memory failures. This work concludes that software-level redundancy can significantly increase the lifespan of remote embedded devices. We recommend this approach for systems where manual reflashing is physically, economically, or technically impractical.

Keywords: *STM32;In-Application Programming;Firmware Resiliency;Bit Flips;Bootloader*

Contents

Acknowledgements	ii
Abstract	iii
List of Figures	vi
List of Tables	vii
Abbreviations	viii
1 Introduction	1
1.1 Background	1
1.2 Objectives	1
1.3 Motivation and Significance	2
2 Related Works	3
3 Design and Implementation	4
3.1 System Overview	4
3.2 System Design	4
3.2.1 Architectural Design	4
3.2.2 Module/Component Design	5
3.2.3 Memory Map Design	6
3.3 Implementation Details	7
3.3.1 Firmware Implementation	7
3.3.2 Firmware Corruption Simulation	11
4 Results and Discussion	13
4.1 Implementation Results	13
4.2 Functional Validation	14
4.2.1 Normal Boot Operation	14
4.2.2 Corruption Detection	15
4.2.3 Autonomous Firmware Recovery	15
4.3 Comparison with the Project Objectives	16
4.4 Discussion	17
4.5 Limitations	18
4.6 Chapter Summary	19
5 Conclusion and Future Enhancements	20
5.1 Conclusion	20
5.2 Limitations	21
5.3 Future Enhancements	22

References	23
Appendix A	24
A.1 Configured Flash Memory Map	24
A.2 Bootloader and Recovery Logic	24
A.3 Functional Validation Procedure	25
A.3.1 Normal Boot Test	25
A.3.2 Injected Corruption Test	26
A.3.3 Recovery Verification Test	26
A.4 Development and Reproduction Checklist	26
A.5 Project Schedule	27

List of Figures

3.1	Quadripartition memory structure	4
3.2	Implemented Architecture	5
3.3	Flowchart of CRC calculation unit.	6
3.4	Application linker script	7
3.5	Bootloader linker script	7
3.6	CRC initialization	8
3.7	Application Validation logic	8
3.8	Erase of Active Application Slot	9
3.9	Reprogramming of Active Application Slot	10
3.10	Page Erasure Flowchart for FPEC. Source: STMicroelectronics (2012)	10
3.11	Flash Program Flowchart for FPEC. Source: STMicroelectronics (2012)	11
3.12	Memory contents before corruption	12
3.13	Memory contents after corruption	12
A.1	Project planning and implementation schedule	27

List of Tables

4.1	Implemented components and their outcomes	14
4.2	Functional validation of the self-healing sequence	16
4.3	Assessment of the project objectives	17
A.1	Configured flash memory regions	24

Abbreviations

CPU Central Processing Unit.

CRC Cyclic Redundancy Check.

FPEC Flash Program and Erase Controller.

HAL Hardware Abstraction Layer.

IAP In-Application Programming.

KU Kathmandu University.

LEO Low Earth Orbit.

OTA Over-the-Air.

SEU Single Event Upset.

UART Universal Asynchronous Receiver-Transmitter.

WDT Watchdog Timer.

Chapter 1

Introduction

1.1 Background

In recent years, embedded systems have been added to many forms of modern critical infrastructure. The types of embedded systems range from remote sensor networks to industrial control systems. One popular type of microcontroller used in many of the applications in the field of embedded systems is the STM32F103C8T6, commonly referred to as the "Blue Pill module". It is cheap, small, and capable of handling a wide variety of tasks, hence the reason for its popularity.

Unlike smartphones, which come with the ability to seamlessly receive Over-the-Air (OTA) firmware patches for recovering from software failures, many embedded systems located in remote or inaccessible locations, such as CubeSats in Low Earth Orbit (LEO), deep field sensor nodes present significant challenges to implementing OTA. Faulty firmware images in such scenarios are incapable of being fixed by a simple remote patch update because there typically will be no possibility of physically accessing the device; therefore, having onboard fault-resistance becomes both necessary and mission-critical.

Embedded devices based on this class of microcontrollers are susceptible to what are known as soft errors or bit flips. Soft errors occur when high-energy cosmic-ray particles or transient electrical noise cause a single bit to flip. These soft errors may cause an otherwise reliable embedded system to behave erratically or completely fail and come to a stop (a state of being bricked) once they corrupt an essential instruction or configuration word of the application program stored in the flash memory.

Recent trends in embedded security and reliability focus on a "Dual-bank" approach to mitigate these risks. However, for low-cost microcontrollers like the STM32F103C8T6, which do not come with hardware support for implementing the dual-bank approach, such solutions are impractical. Addressing this problem, this project focuses on utilizing the internal flash memory of the microcontroller to create a backup copy of the original firmware, hence providing a way for self-correction.

1.2 Objectives

The specific technical objectives of this project are as follows:

- To design a flash memory map that partitions the device into three logical regions: the bootloader, an active application program, and a recovery image.

- To implement a CRC-based integrity verification algorithm within the custom bootloader to detect bit-level corruption at power-on.
- To develop a firmware recovery routine leveraging IAP to autonomously restore a corrupted application from a stored recovery image.
- To empirically demonstrate the system’s self-healing capability by simulating firmware corruption and validating successful recovery.

1.3 Motivation and Significance

The motivation for this project stems from the extreme reliability requirements of "New Space" missions and CubeSat technology. Unlike terrestrial embedded systems, microcontrollers deployed in LEO are exposed to high-energy cosmic rays and solar particles. These environmental factors frequently cause Single Event Upsets (SEUs), commonly known as "bit flips," in the flash memory and registers of the microcontroller.

For a CubeSat, a single bit flip in the application firmware can lead to "Loss of Mission". Since physical access to the satellite is impossible once launched, and radio communication (uplink) is often limited by narrow bandwidth and specific ground-station passes, the satellite must be capable of autonomous survival. This project explores a software-defined approach to radiation resilience, allowing a budget-friendly STM32F103C8T6, which is not inherently radiation-hardened, to survive in harsh environments by detecting and repairing its own code.

Standard reliability measures like Watchdog Timer (WDT) can recover a system from a software hang, but they cannot repair the underlying code if the binary itself is corrupted. If a bit flip occurs in the application sector, a standard reboot will simply restart the corrupted code, leading to a "boot loop" or permanent failure. This project addresses this drawback by moving beyond "system reset" to "system repair." By using IAP to overwrite corrupted sectors with a protected "Backup Image", the system ensures that the firmware integrity is restored to a known-good state without requiring a command from Earth.

This project is significant because it paves the way for access to resilient satellite sub-systems, proving that intelligent software design can compensate for the hardware vulnerabilities of low-cost microcontrollers in space-like environments.

Chapter 2

Related Works

The foundation of modern firmware management often relies on the separation of boot and application memory regions. According to Zhou (2015), implementing a centralized firmware upgrader using IAP methodology can significantly reduce the complexity of managing multi-component robotic systems. Zhou’s work focused on a master-upgrader architecture using an STM32F4 series microcontroller to autonomously flash slave devices. A key contribution of this project was the development of the ”Smart Link” protocol, which integrated CRC-8 and CRC-16 checksums to ensure that raw firmware data remained uncorrupted during transmission. This highlights the critical nature of integrity checks in IAP systems; however, Zhou’s implementation was primarily designed for manual upgrades and version control rather than autonomous self-healing.

In a more recent study, Neves et al. (2024) introduced an innovative paradigm for firmware updates during runtime. Their research focused on minimizing system downtime—achieving recovery times of less than 10 ms—by replacing only specific code segments rather than the entire firmware. This was made possible through the use of static and absolute function addressing, which allowed the IAP controller to target specific memory blocks without affecting the rest of the running application. While Neves et al. (2024) successfully demonstrated that partial updates are ideal for bandwidth-limited channels and high-uptime requirements, their work noted a limitation: the success of such updates is highly dependent on power stability and requires external backup power (like supercapacitors) to maintain integrity during the write cycle.

The proposed project builds upon these established ideas of IAP and memory partitioning but focuses specifically on the problem of firmware corruption and ”bit flips”. While Zhou (2015) solved the problem of delivering new firmware and Neves et al. (2024) solved the problem of runtime updates, this project aims to create an autonomous ”Self-Healing” loop. By combining the CRC-based verification logic emphasized by Zhou and the memory partitioning/absolute addressing techniques discussed by Neves, this project will implement a system where the device monitors its own health. Unlike previous works that rely on a host to send new data, this system will autonomously restore its state from an internal ”Backup Image” partition, thereby providing a robust solution for devices deployed in harsh or remote environments.

Chapter 3

Design and Implementation

This chapter describes the design and implementation of the autonomous firmware recovery system. It details the memory organization of the microcontroller, the IAP logic utilized for self-remediation, and the specific structural considerations that enable the Self-Healing Architecture.

3.1 System Overview

The Self-Healing Architecture is an embedded security framework designed to detect and repair firmware corruption without external intervention. The system is built on a quadripartition memory structure consisting of:

- **Bootloader:** A protected software layer responsible for integrity verification and jumping to the application.
- **Application Header:** The region of the memory that consists of the **32-bit** CRC value of the original firmware along with the firmware size.
- **Active Application Slot:** The primary region where the operational firmware runs and is monitored for bit-flips or unauthorized changes.
- **Backup Image Slot:** A read-only “Golden Image” of the firmware used as a reference for autonomous restoration.

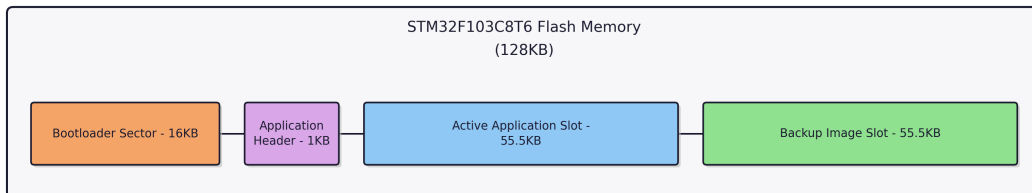


Figure 3.1: Quadripartition memory structure

3.2 System Design

3.2.1 Architectural Design

The system employs a “Secure-Boot” architecture to ensure firmware authenticity. The memory is logically partitioned such that the **Bootloader** acts as the root of trust. It evaluates the integrity of the **Active Application Slot** before every execution cycle, ensuring that corrupted instructions are never processed by the Central Processing Unit (CPU).

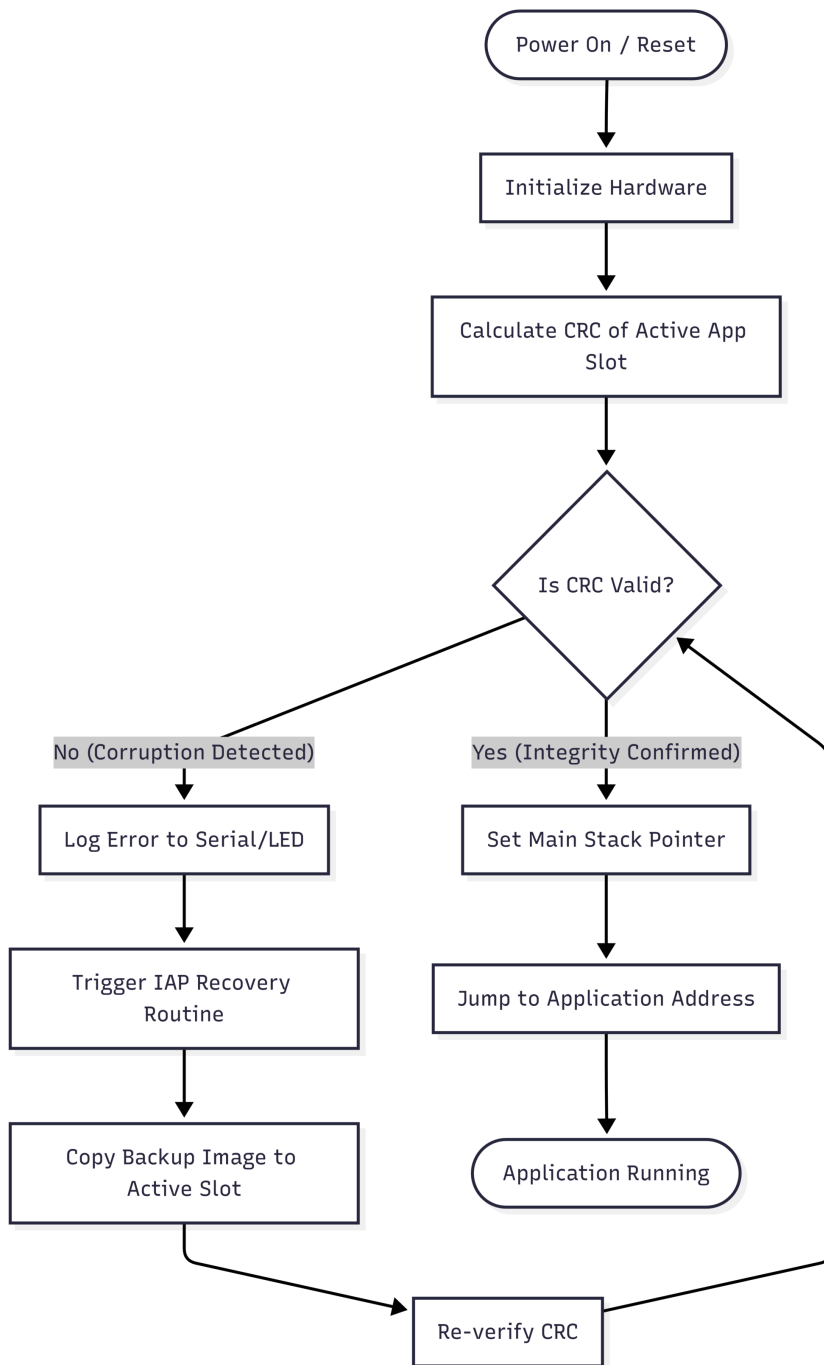


Figure 3.2: Implemented Architecture

3.2.2 Module/Component Design

The system logic is divided into four core modules:

- **Verification Module:** Utilizes the hardware CRC unit to generate a signature of the current application memory. The flowchart of the hardware CRC calculation unit is shown in Figure 3.3.

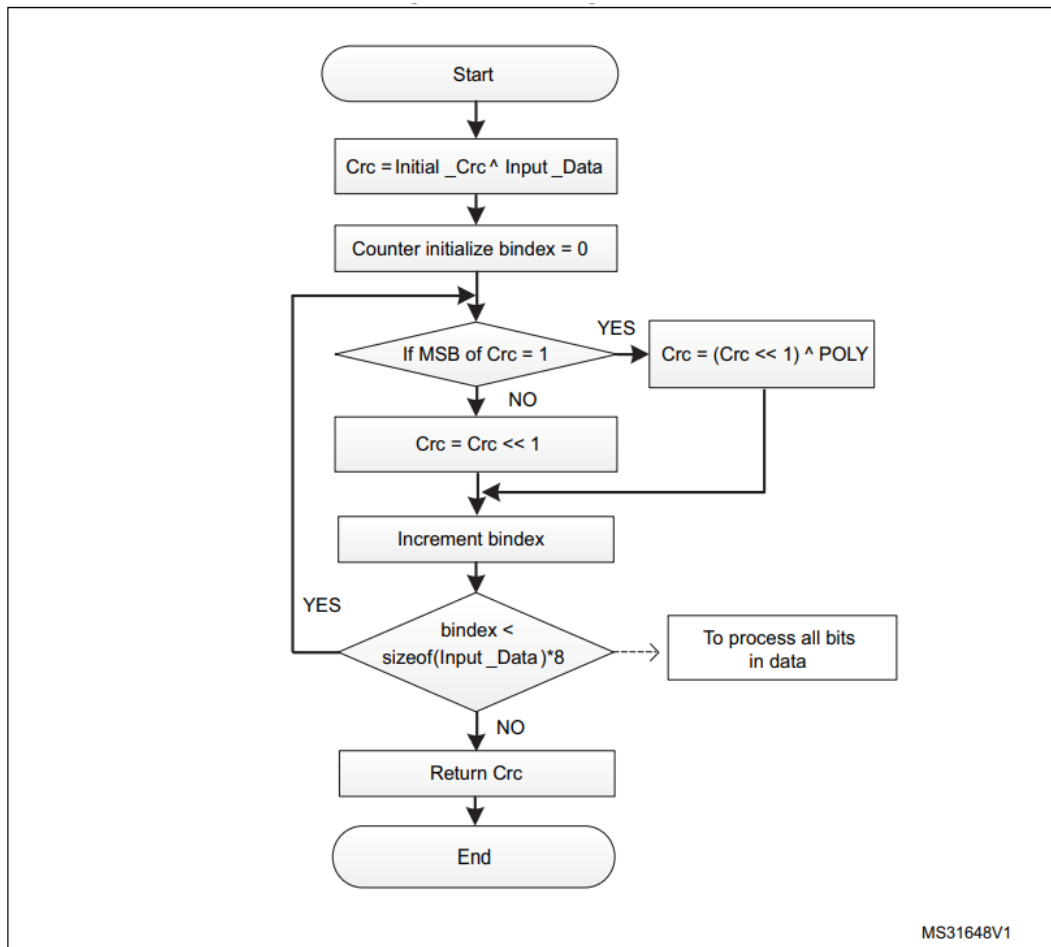


Figure 3.3: Flowchart of CRC calculation unit. Source: STMicroelectronics (2022)

- **Decision Engine:** Compares the calculated signature against the “Golden Hash” stored in the Application Header Slot.
- **IAP Recovery Module:** Orchestrates the flash erase and write operations by interacting with the Flash Program and Erase Controller (FPEC) to restore the system state.
- **Telemetry Module:** Reports health status and recovery logs via the Universal Asynchronous Receiver-Transmitter (UART) serial interface for monitoring.

3.2.3 Memory Map Design

The internal flash of the STM32 is mapped to accommodate the self-healing logic. The **Bootloader** occupies the first 16KB, followed by the **Application Header** (1KB), **Active Slot** (55.5KB), and the **Backup Slot** (55.5KB) as shown in Figure 3.1. Specific memory addresses are hard-coded through the **linker script** as shown in Figure 3.4 and Figure 3.5 to ensure the IAP driver targets the correct sectors during a write cycle.

```

1 MEMORY
2 {
3     RAM        (xrw)    : ORIGIN = 0x20000000,    LENGTH = 20K
4     APP_HEADER (rx)      : ORIGIN = 0x8004000,    LENGTH = 1K
5     FLASH      (rx)      : ORIGIN = 0x8004400,    LENGTH = 55K + 512
6 }

```

Figure 3.4: Application linker script

```

3 /* Memories definition */
2 MEMORY
1 {
47 RAM        (xrw)    : ORIGIN = 0x20000000,    LENGTH = 20K
1 FLASH      (rx)      : ORIGIN = 0x8000000,    LENGTH = 16K
2 }

```

Figure 3.5: Bootloader linker script

3.3 Implementation Details

3.3.1 Firmware Implementation

The implementation is written in embedded C with the help of Hardware Abstraction Layer (HAL) to maintain a small memory footprint and high execution speed.

3.3.1.1 Bootloader

The bootloader is the entry point of the system, which is stored at the start address of the memory (**0x0800 0000**) and is allocated a maximum size of 16KB as shown in Figure 3.5. It initializes the CRC calculation unit as shown in Figure 3.6. The bootloader then validates the application integrity as follows:

1. Fetch the application header start address that stores the CRC value of the original application firmware.
2. Fetch the active application start address.
3. Calculate the CRC value of the active application.
4. Compare the calculated CRC value against the value stored in the application header.
5. Return error code **3** if the CRC values do not match. Else, return success code **0**.


```

1 static void MX_CRC_Init(void)
2 {
3
4     /* USER CODE BEGIN CRC_Init 0 */
5
6     /* USER CODE END CRC_Init 0 */
7
8     /* USER CODE BEGIN CRC_Init 1 */
9
10    /* USER CODE END CRC_Init 1 */
11    hcrc.Instance = CRC;
12    if (HAL_CRC_Init(&hcrc) != HAL_OK)
13    {
14        Error_Handler();
15    }
16    /* USER CODE BEGIN CRC_Init 2 */
17    __HAL_RCC_CRC_CLK_ENABLE();
18    /* USER CODE END CRC_Init 2 */
19
20 }

```

Figure 3.6: CRC initialization

```

1 int is_application_not_valid(void) {
2     const app_header_t *app_header = (const app_header_t *) APP_HEADER_START_ADDR;
3
4     uint32_t *application_start_addr = (uint32_t *) APP_START_ADDR;
5     uint32_t num_of_words = app_header->app_size / 4;
6
7     uint32_t crc = HAL_CRC_Calculate(&hcrc, application_start_addr, num_of_words);
8     if (crc != app_header->crc) {
9         return 3;
10    }
11
12    return 0;
13 }

```

Figure 3.7: Application Validation logic

3.3.1.2 Flash Driver

The Flash Driver is a part of the Bootloader that communicates with the FPEC and is responsible for reprogramming the Active Application Slot by retrieving the original application firmware from the Backup Slot (stored from memory location **0x0801 2200**). It performs the following steps in sequence:

1. Erasure of Active Application Slot:
 - Fetch the application header and calculate the total number of pages to be erased with the help of the application size stored in the application header. Each page is **1KB** in size STMMicroelectronics (2012).

- Erase the calculated number of pages from the flash memory starting from the base address of the Active Application Slot.
- Check each erased memory address for successful erasure.

```

1 void erase_application_pages(void) {
2     const app_header_t *app_header = (const app_header_t *) APP_HEADER_START_ADDR;
3     uint32_t app_size = app_header->app_size;
4     uint8_t number_of_pages = (uint8_t)ceil(app_size / 1024.0);
5     for (uint8_t i = 0; i < number_of_pages; i++) {
6         flash_erase(APP_START_ADDR + (i * 1024));
7     }
8
9     /* Check every erased address */
10    char message[100];
11    uint32_t *application_start_addr = (uint32_t *) APP_START_ADDR;
12    uint32_t number_of_words = ceil(app_size / 4.0);
13    do {
14        uint32_t value_at_address = *(application_start_addr + number_of_words);
15        if (value_at_address != 0xFFFFFFFF) {
16            sprintf(message, "Error erasing application memory: %p\r\n", (application_start_addr + number_of_words));
17            HAL_UART_Transmit(&huart1, (uint8_t *)message, strlen(message), HAL_MAX_DELAY);
18        }
19        number_of_words--;
20    } while(number_of_words != 0);
21    sprintf(message, "Application memory erase operation completed!!!\r\n");
22    HAL_UART_Transmit(&huart1, (uint8_t *)message, strlen(message), HAL_MAX_DELAY);
23 }

```

Figure 3.8: Erasure of Active Application Slot

2. Reprogramming of the Active Application Slot:

- Fetch the application header and calculate the total number of words to be written with the help of the application size stored in the application header. Each word is **32-bit** in length.
- Fetch a word from the Backup Slot and divide it into upper half 16-bit and lower half 16-bit.
- Write the half-words one at a time into the desired Active Application Slot.
- Repeat second and third step until the number of words to be written is zero.

```

1 void write_application_into_flash(void) {
2     const app_header_t *app_header = (const app_header_t *) APP_HEADER_START_ADDR;
3     const uint32_t *app_bkp_start_addr = (const uint32_t *) APP_BKP_START_ADDR;
4     uint32_t app_size = app_header->app_size;
5     uint32_t number_of_words = (uint32_t)ceil(app_size / 4.0);
6
7     uint32_t bkp_data;
8     uint16_t upper_half;
9     uint16_t lower_half;
10    for (uint32_t i = 0; i < number_of_words; i++) {
11        bkp_data = *(app_bkp_start_addr + i);
12        upper_half = (bkp_data >> 16);
13        lower_half = (bkp_data >> 0);
14
15        flash_program(APP_START_ADDR + (i * 4), lower_half);
16        flash_program(APP_START_ADDR + (i * 4) + 2, upper_half);
17    }
18
19    char message[100];
20    sprintf(message, "Application rewritten to flash\r\n");
21    HAL_UART_Transmit(&huart1, (uint8_t *)message, strlen(message), HAL_MAX_DELAY);
22 }

```

Figure 3.9: Reprogramming of Active Application Slot

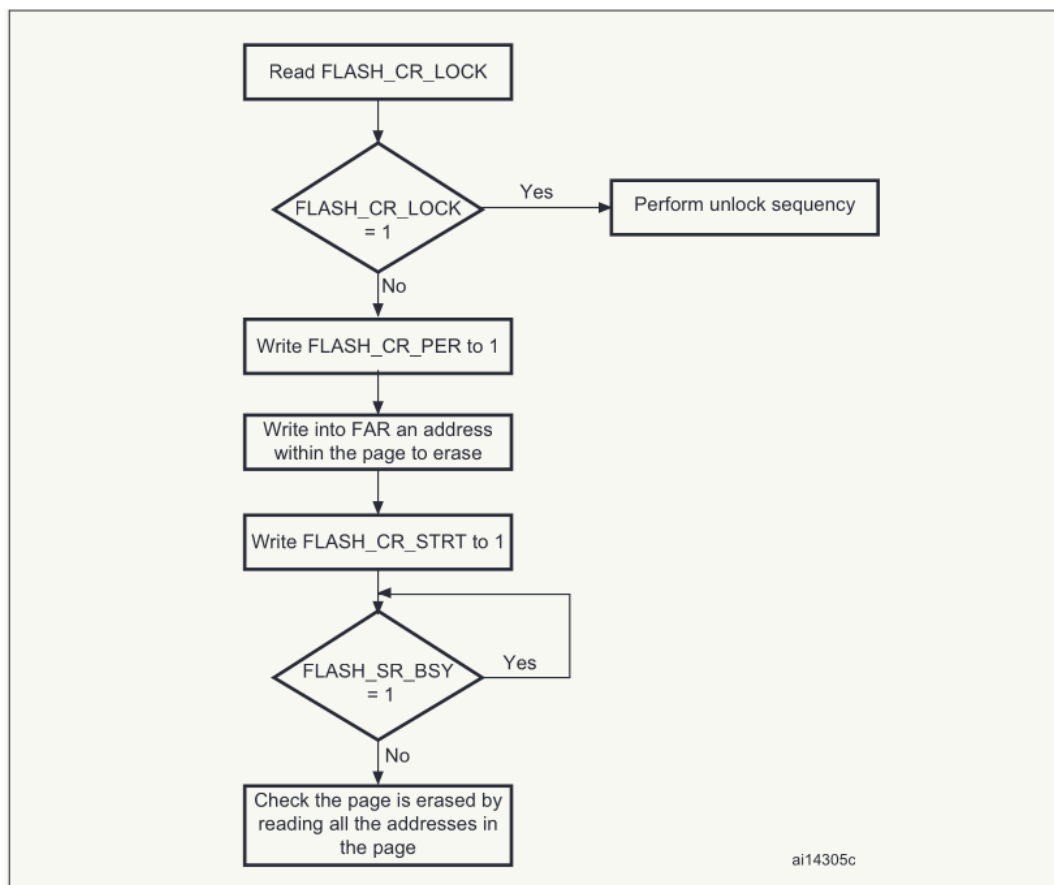


Figure 3.10: Page Erasure Flowchart for FPEC. Source: STMicroelectronics (2012)

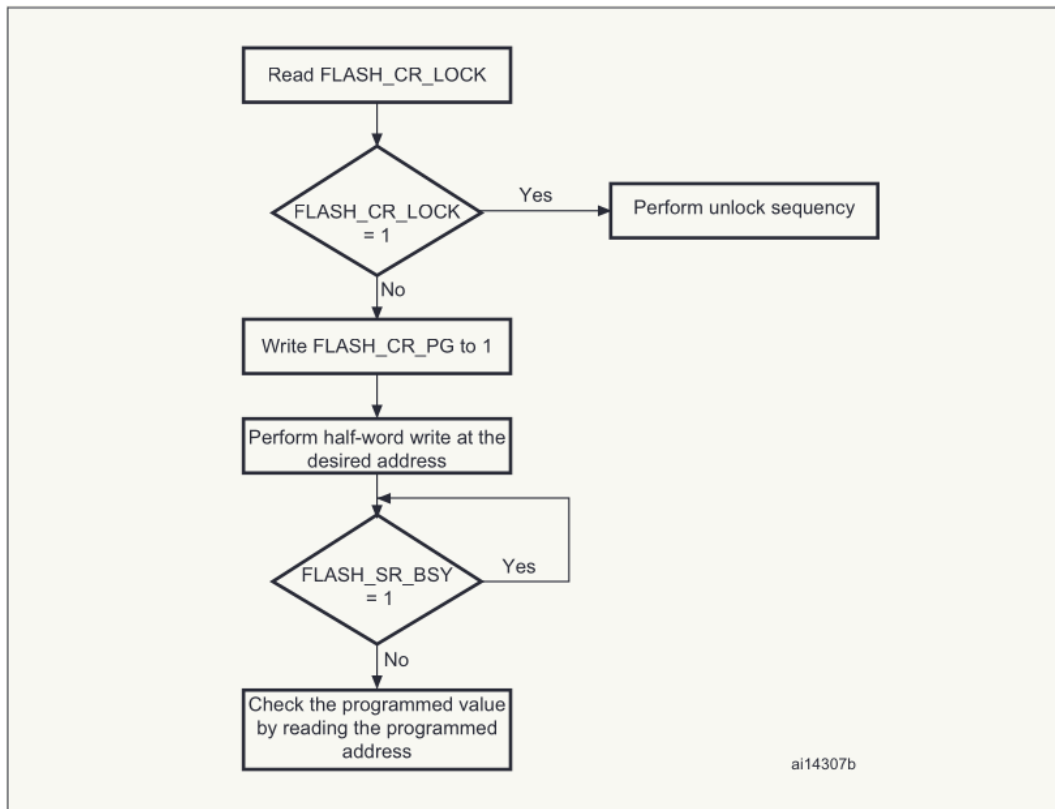


Figure 3.11: Flash Program Flowchart for FPEC. Source: STMicroelectronics (2012)

3.3.2 Firmware Corruption Simulation

The corruption of the firmware is simulated by manually changing the data stored at any address of the Active Application Slot. This is done with the help of **STM32 CubeProgrammer**; a software that is used to flash binary files into the flash memory. It also allows us to view the data stored at every single address of the flash memory, and edit the data stored.

As shown in Figure 3.12, the **4 bytes** data starting at memory location **0x08000000** is **0x20005000**. The data is then manually corrupted to **0x20005FFF** as shown in Figure 3.13.

Address	0	4	8	C
0x08004400	20005000	08004909	08004839	08004841
0x08004410	08004849	08004851	08004859	00000000
0x08004420	00000000	00000000	00000000	08004861
0x08004430	0800486D	00000000	08004879	08004885
0x08004440	08004951	08004951	08004951	08004951
0x08004450	08004951	08004951	08004951	08004951
0x08004460	08004951	08004951	08004951	08004951
0x08004470	08004951	08004951	08004951	08004951
0x08004480	08004951	08004951	08004951	08004951
0x08004490	08004951	08004951	08004951	08004951
0x080044A0	08004951	08004951	08004951	08004951
0x080044B0	08004951	08004951	08004951	08004951
0x080044C0	08004951	08004951	08004951	08004951
0x080044D0	08004951	08004951	08004951	08004951
0x080044E0	08004951	08004951	08004951	00000000

Figure 3.12: Memory contents before corruption

0x08004400	20005FFF	08004909	08004839	08004841
0x08004410	08004849	08004851	08004859	00000000
0x08004420	00000000	00000000	00000000	08004861
0x08004430	0800486D	00000000	08004879	08004885
0x08004440	08004951	08004951	08004951	08004951
0x08004450	08004951	08004951	08004951	08004951
0x08004460	08004951	08004951	08004951	08004951
0x08004470	08004951	08004951	08004951	08004951
0x08004480	08004951	08004951	08004951	08004951
0x08004490	08004951	08004951	08004951	08004951
0x080044A0	08004951	08004951	08004951	08004951
0x080044B0	08004951	08004951	08004951	08004951
0x080044C0	08004951	08004951	08004951	08004951
0x080044D0	08004951	08004951	08004951	08004951
0x080044E0	08004951	08004951	08004951	00000000
0x080044F0	00000000	00000000	00000000	00000000

Figure 3.13: Memory contents after corruption

Chapter 4

Results and Discussion

This chapter presents the results obtained from implementing and validating the autonomous firmware recovery architecture described in the previous chapter. The evaluation focused on the four project objectives: partitioning the internal flash memory, verifying the active firmware at start-up, restoring a corrupted image through IAP, and demonstrating the complete self-healing sequence after an injected fault. The discussion also relates the implemented system to the approaches reviewed in the related works and identifies the limitations of the current prototype.

4.1 Implementation Results

The implementation produced a boot-controlled firmware architecture in which the application is not executed until its integrity has been checked. The flash layout was separated into four logical regions: the bootloader, the application header, the active application slot, and the backup image slot. The application header stores the size and reference CRC of the known-good firmware. These values allow the bootloader to verify only the programmed application region and to determine the number of flash pages and words needed during recovery.

The main functional outcomes of the implementation are summarized in Table 4.1.

Table 4.1: Implemented components and their outcomes

Component	Implemented function	Result
Flash memory map	Separates the recovery logic, metadata, active firmware, and backup image	Prevents normal application execution from overlapping the recovery regions
Application header	Stores the firmware size and reference CRC	Provides the metadata required for bounded verification, erasure, and copying
Verification module	Calculates the active image CRC using the hardware peripheral	Distinguishes an unchanged image from an image modified by fault injection
Decision engine	Compares the calculated and stored CRC values	Selects either normal application start-up or the recovery path
IAP recovery module	Erases the affected active pages and copies the backup image into the active slot	Restores the active image to the known-good firmware state
Application handover	Transfers control to the active firmware after validation	Allows execution only after a successful integrity check

4.2 Functional Validation

The prototype was evaluated by observing the bootloader under both an intact firmware condition and a deliberately corrupted firmware condition. This fault-injection method represents the effect of a persistent bit-level change in the active application image. It provides a repeatable way to test the recovery logic, although it does not reproduce the physical radiation process that causes an SEU.

4.2.1 Normal Boot Operation

For the normal boot test, the active application was kept identical to the known-good backup image. At reset, the bootloader read the firmware length from the application header and calculated the CRC over the active slot. The calculated value matched the reference value stored in the header. The validation function therefore returned its success status, the flash recovery routine was bypassed, and control was transferred to the active application. This result confirms that the integrity check does not unnecessarily rewrite a valid application during every boot cycle.

4.2.2 Corruption Detection

The fault-injection test altered data within the active application region while leaving the reference CRC and backup image unchanged. On the next reset, the hardware CRC calculation produced a value different from the reference stored in the application header. The validation function returned the mismatch status described in the implementation chapter, preventing the bootloader from transferring control to the corrupted application. The test therefore demonstrated that a persistent modification within the verified region can be detected before the modified instructions are executed.

4.2.3 Autonomous Firmware Recovery

After detecting the mismatch, the bootloader invoked the flash driver without requiring a host command or a replacement image over a communication link. The driver used the firmware size stored in the header to determine the affected flash pages, erased the active application region, and copied the known-good firmware from the backup slot. Because the STM32F103 flash interface is programmed in half-words, each 32-bit word from the backup image was divided into two 16-bit writes during this operation.

The restored active image was then subjected to the same integrity check. Its calculated CRC matched the reference value, after which the bootloader transferred control to the application. This second verification is important: it confirms that the copy operation produced the expected image rather than assuming that page erasure and programming completed correctly. The observed boot sequence is summarized in Table 4.2.

Table 4.2: Functional validation of the self-healing sequence

Test condition	Bootloader response	Outcome
Active image unchanged	Calculated and reference CRC values match	Recovery is skipped and the application starts normally
Active image deliberately modified	Calculated and reference CRC values differ	Application start-up is blocked and recovery is initiated
Recovery in progress	Active pages are erased and the backup image is copied into the active slot	Corrupted contents are replaced without external firmware transfer
Restored image checked	Recalculated CRC matches the reference	The restored application is accepted and executed

4.3 Comparison with the Project Objectives

Table 4.3 compares the implemented results with the objectives stated in the introduction. The assessment is based on the implemented modules and functional tests described above.

Table 4.3: Assessment of the project objectives

Objective	Evidence from implementation and testing	Status
Partition flash into bootloader, application, and recovery regions	Linker-defined regions were implemented for the bootloader, header, active application, and backup image	Achieved
Detect bit-level firmware corruption at power-on	The bootloader calculated the active image CRC and detected the injected change before application handover	Achieved
Restore corrupted firmware autonomously using IAP	The flash driver erased and reprogrammed the active slot from the internal backup image	Achieved
Demonstrate self-healing through simulated corruption	The complete detect–recover–revalidate–execute sequence was exercised using a deliberately modified active image	Achieved

The memory design uses four regions rather than the three broad regions named in the first objective because the application metadata is isolated in a separate header. This is a refinement of the original objective rather than a change in its purpose: the active and recovery images remain distinct, while the dedicated header makes verification and size-bounded recovery possible.

4.4 Discussion

The results show that the prototype moves beyond reset-based fault recovery. A WDT can restart a processor after a software hang, but it cannot repair instructions that remain corrupted in non-volatile memory. In the implemented architecture, every reset returns control to the bootloader, which checks the application and replaces it when necessary. The recovery decision is therefore based on the contents of the firmware rather than only on whether the processor is responsive.

The system also differs from the firmware-management approaches discussed in the related works. The centralized upgrader presented by Zhou (2015) uses IAP and checksum mechanisms to deliver firmware to other devices. In contrast, the present prototype keeps its recovery source on the same microcontroller and does not require a master device during repair. The partial-update method described by Neves et al. (2024) reduces downtime by replacing selected code segments at runtime. The implemented system instead restores the complete active image during boot. This requires more storage and potentially more write time, but it simplifies the recovery decision and is suitable when the priority is returning to a single known-good state.

Using the hardware CRC peripheral keeps the integrity-checking logic small and avoids implementing the calculation entirely in software. The reference value also provides a compact representation of the expected firmware. However, a CRC is an error-detection mechanism and not a cryptographic authentication mechanism. It is appropriate for detecting accidental changes such as the simulated bit corruption considered in this project, but it does not protect against an attacker who can deliberately modify both the firmware and its stored reference value.

The internal backup removes dependence on an immediate OTA connection, which is valuable for remote sensor nodes and CubeSat subsystems with limited communication opportunities. This independence comes at the cost of flash capacity: two copies of the application and a bootloader must fit on the same device. Consequently, the maximum application size is smaller than in a conventional single-image layout, and the linker configuration must match the actual flash capacity of the selected microcontroller.

4.5 Limitations

The functional results demonstrate the intended recovery sequence, but the current prototype has several limitations:

- **Simulated fault injection:** The tests reproduce persistent firmware modification but do not quantify behavior under a radiation source or across a statistical population of naturally occurring SEUs.
- **Single recovery image:** Corruption of the backup image, application header, or bootloader itself cannot be repaired by copying the backup into the active slot. These regions require additional protection or redundancy.
- **Power interruption during recovery:** Loss of power while a page is being erased or programmed may leave the active image incomplete. The next reset can attempt recovery again, provided that the bootloader, header, and backup image remain intact.
- **Flash capacity and endurance:** Maintaining two firmware images reduces the space available to the application. Repeated recovery cycles also consume finite flash erase/program endurance.
- **Integrity rather than authenticity:** The CRC detects accidental corruption but does not provide cryptographic verification or protection against intentional modification.
- **No quantitative timing or energy characterization:** The present evaluation establishes functional behavior but does not report boot-time overhead, recovery duration, or energy consumption. These measurements are needed to assess suitability for strict real-time and low-power deployments.

4.6 Chapter Summary

The implementation successfully demonstrated the core self-healing loop. An intact application passed verification and started without being rewritten, whereas an injected modification caused a CRC mismatch, blocked execution, and triggered restoration from the internal backup image. Revalidation of the restored image completed the recovery sequence before application handover. The results satisfy the functional objectives of the project while also showing that backup protection, power-failure tolerance, memory capacity, and quantitative performance evaluation remain important areas for further work.

Chapter 5

Conclusion and Future Enhancements

This project presented the design, implementation, and functional validation of a self-healing bootloader for the STM32F103C8T6 microcontroller. The primary objective was to enable autonomous detection and recovery of corrupted application firmware in deployment scenarios where physical access or continuous communication is unavailable. By integrating firmware integrity verification with an automatic recovery mechanism, the proposed architecture improves system reliability without requiring external recovery tools or user intervention.

5.1 Conclusion

The implemented architecture organises the microcontroller's internal flash memory into four logical regions: the bootloader, an application header containing firmware metadata, the active application, and a backup firmware image. During every system reset, the bootloader reads the expected firmware size and CRC value from the application header, computes a new CRC using the STM32 hardware CRC peripheral, and compares the calculated result with the stored reference value. When both values match, execution proceeds normally. If a mismatch is detected, the bootloader initiates an automatic recovery procedure that erases the corrupted application, restores the backup firmware through in-application programming (IAP), performs a second integrity check, and finally transfers control to the restored application.

Functional testing demonstrated that intentionally corrupted firmware was consistently detected and successfully restored without external intervention. The results confirm that a software-based recovery mechanism can significantly improve the fault tolerance of a resource-constrained, single-bank microcontroller beyond the protection provided by conventional reset or watchdog mechanisms. Since the entire recovery process is performed using only on-chip resources, the proposed approach is well suited for remote embedded systems, unattended IoT devices, and CubeSat subsystems where maintenance opportunities are limited.

The project successfully achieved its primary objectives:

- Designing a linker-script-based flash memory layout that separates the bootloader, firmware metadata, active application, and backup firmware.
- Implementing firmware integrity verification using the STM32 hardware CRC peripheral.
- Developing an in-application programming (IAP) mechanism to erase and restore the active firmware from the internal backup image.

- Demonstrating the complete detect–recover–revalidate–execute workflow through controlled firmware corruption experiments.

Overall, the project demonstrates that careful flash memory partitioning, combined with firmware integrity verification and an autonomous recovery process, can transform a conventional bootloader into a reliable self-healing mechanism capable of restoring damaged firmware after a system reset.

5.2 Limitations

Although the proposed system was successfully validated, several limitations remain.

The recovery mechanism protects only the active application firmware. Corruption of the bootloader, the application header, or the backup firmware cannot be repaired by the current implementation and would leave the device unrecoverable.

Maintaining a complete backup copy of the firmware requires additional internal flash memory equal to the application size. For resource-constrained microcontrollers, this reduces the memory available for application development.

Each recovery cycle involves flash erase and programming operations. Since flash memory has a limited write endurance, repeated recovery events gradually reduce the lifespan of the storage cells. The present implementation does not include wear-levelling or erase-cycle monitoring.

Unexpected power loss during the erase or programming process may leave the active firmware incomplete or inconsistent. Although recovery can be attempted again after a reset, successful restoration depends on the bootloader, application header, and backup firmware remaining intact.

Firmware integrity verification currently relies solely on CRC comparison. While CRC is highly effective for detecting accidental data corruption, it does not provide cryptographic authentication or protect against intentional firmware modification.

Finally, validation was limited to controlled laboratory experiments in which firmware corruption was deliberately introduced. Performance metrics such as recovery time, boot-time overhead, energy consumption, and resilience under radiation-induced faults were not experimentally evaluated.

5.3 Future Enhancements

Several improvements can further increase the robustness and applicability of the proposed architecture.

Protection should be extended beyond the application firmware to include the bootloader and application header. Hardware write protection for the bootloader, together with redundant copies of the firmware header using sequence numbers or version control, would improve resilience against metadata corruption.

Replacing the single backup firmware with multiple verified backup copies would provide greater fault tolerance and enable recovery even if one backup image becomes corrupted. Furthermore, page-level recovery, rather than complete firmware restoration, could reduce both recovery time and flash wear.

A transactional recovery mechanism capable of recording recovery progress would allow interrupted recovery operations to resume safely after unexpected power loss. Integration with brown-out detection and energy-storage circuitry could further improve reliability during flash programming.

Replacing CRC with cryptographic hash functions and digital signatures would enable secure boot functionality, ensuring both firmware integrity and authenticity by preventing unauthorised firmware execution.

Future work should also include comprehensive performance evaluation, including measurements of boot overhead, recovery latency, flash memory wear, and energy consumption under realistic deployment conditions. These measurements would provide a quantitative assessment of the architecture's suitability for mission-critical embedded systems.

Finally, integrating telemetry support would allow recovery events to be logged for later analysis, while an authenticated over-the-air (OTA) update mechanism could securely replace the backup firmware whenever a communication link becomes available. These enhancements would transform the current proof-of-concept implementation into a more practical and deployment-ready self-healing firmware platform for long-duration autonomous embedded applications.

References

- Neves, B. P., Santos, V. D. N., and Valente, A. (2024). Innovative firmware update method to microcontrollers during runtime. *Electronics*, 13(7).
- STMicroelectronics (2012). *PM0075: STM32F10xxx Flash Memory Programming Manual*. STMicroelectronics, revision 2 edition. Document ID 17863 Rev 2.
- STMicroelectronics (2022). *AN4187: Using the CRC Peripheral on STM32 Microcontrollers*. STMicroelectronics, rev. 2 edition. Application Note.
- Zhou, W. (2015). *COMPACT FIRMWARE UPGRADER FOR ROBOT SYSTEM BASED ON IAP METHODOLOGY*. PhD thesis, Cornell University.

Appendix A

A.1 Configured Flash Memory Map

Table A.1 summarizes the logical flash regions used by the self-healing firmware architecture. The addresses are controlled by the bootloader and application linker scripts. Before using this layout on another board, the available flash capacity and page size must be verified against the specific microcontroller fitted to that board.

Table A.1: Configured flash memory regions

Region	Start address	Allocated size	Purpose
Bootloader	0x08000000	16 KB	Initializes the required peripherals, verifies the active image, performs recovery, and transfers control to valid firmware.
Application header	0x08004000	1 KB	Stores recovery metadata, including the reference CRC and application size.
Active application	0x08004400	55.5 KB	Contains the firmware executed after successful integrity verification.
Backup image	0x08012200	55.5 KB	Contains the known-good firmware copied into the active slot when corruption is detected.

The application size stored in the header is used to bound both the CRC calculation and the recovery operation. This prevents the bootloader from checking or rewriting unused flash beyond the programmed application image.

A.2 Bootloader and Recovery Logic

The following pseudocode summarizes the implemented boot decision sequence. It is intended as a compact reference to the detailed flowcharts and source extracts presented in the design and implementation chapter.

```
initialize required hardware
read firmware size and reference CRC from application header
```

```

calculated_crc = CRC(active_application, firmware_size)

if calculated_crc equals reference_crc then
    transfer control to active application
else
    pages = number of pages occupied by firmware_size
    erase pages in active application slot

    for each 32-bit word in the backup image do
        program lower 16-bit half-word into active slot
        program upper 16-bit half-word into active slot
    end for

    restored_crc = CRC(active_application, firmware_size)

    if restored_crc equals reference_crc then
        transfer control to restored application
    else
        remain in a safe error state
    end if
end if

```

The final integrity check is necessary because successful entry into the flash programming routine does not by itself guarantee that every erase and write operation produced the expected image.

A.3 Functional Validation Procedure

The self-healing sequence can be reproduced using the following test cases. Diagnostic status information may be observed through the UART interface or a debugger, depending on the firmware configuration.

A.3.1 Normal Boot Test

1. Program the bootloader, application header, active application, and known-good backup image into their assigned regions.
2. Reset or power-cycle the microcontroller.
3. Confirm that the calculated active-image CRC matches the value stored in the application header.
4. Confirm that the recovery routine is bypassed and the active application starts normally.

A.3.2 Injected Corruption Test

1. Begin with the valid memory state used for the normal boot test.
2. Modify one or more bytes within the active application region while leaving the application header and backup image unchanged.
3. Reset the microcontroller and confirm that the active-image CRC differs from the reference value.
4. Confirm that control is not transferred to the corrupted application and that the recovery routine is invoked.

A.3.3 Recovery Verification Test

1. Allow the bootloader to erase the active application pages and copy the firmware from the backup slot.
2. Recalculate the CRC of the restored active image.
3. Confirm that the restored value matches the reference CRC.
4. Confirm that control is transferred to the restored application.
5. Reset the device once more and confirm that it follows the normal boot path without repeating recovery.

A.4 Development and Reproduction Checklist

The following items are required to reproduce the prototype safely:

- An STM32F103C8T6 development board with sufficient verified flash capacity for the configured memory regions.
- An ST-Link programmer/debugger for initial programming, memory inspection, and controlled fault injection.
- A compatible ARM embedded C toolchain and the STM32 HAL libraries used by the firmware.
- Linker scripts that place the bootloader, active application, and backup image at the configured addresses without overlap.
- A correctly generated application header containing the exact firmware size and matching reference CRC.
- A serial adapter or debugger for observing recovery status and error conditions during validation.
- A stable power source, particularly during flash erase and program operations.

Before fault injection, a copy of the valid firmware and its metadata should be retained on the development host. The injected modification must be restricted to the active application slot; modifying the bootloader, header, or backup slot would test a failure condition outside the recovery scope of the current prototype.

A.5 Project Schedule

Figure A.1 presents the project schedule covering proposal development, background research, implementation, testing, debugging, and documentation.

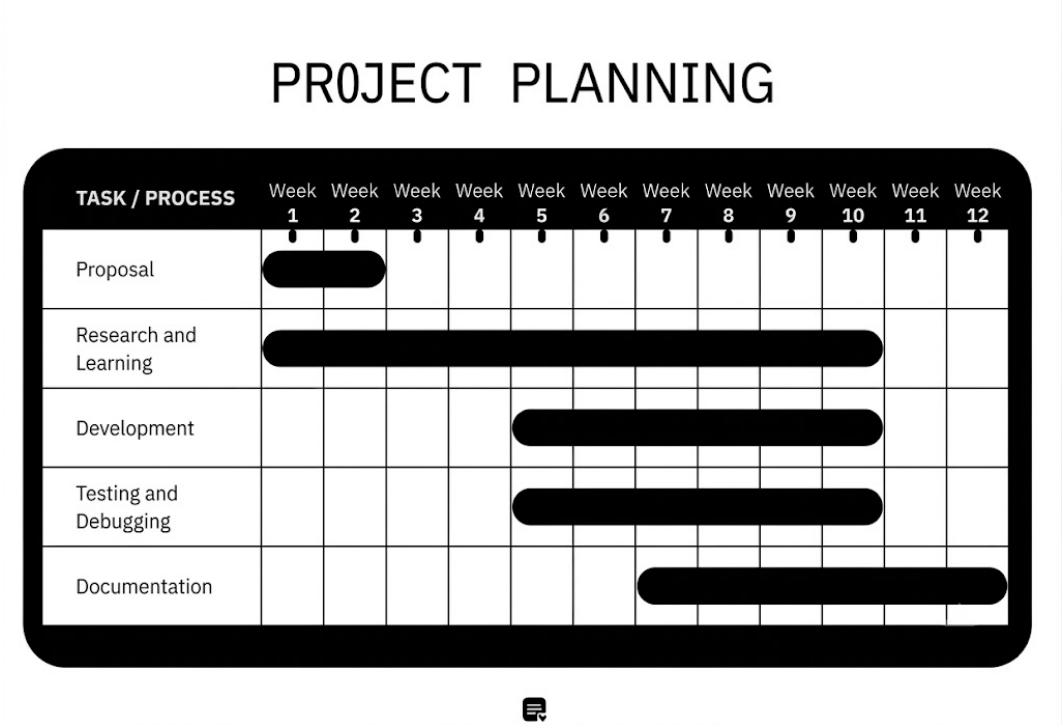


Figure A.1: Project planning and implementation schedule